

Anfängerpraktikum: Entwicklung eines Computerspiels

Leonard Gallus, Paul Horn & Eric Waldherr

Gliederung

- Storyboard
- Use Case
- Anforderungsanalyse
- Architektur
- Implementation
- Prozessentwicklung
- Zusammenfassung



Storyboard

- Ziel des Spiels
- Wie man spielt
- Gegnertypen
- Grafik und Audio
- Geschichte des Spiels



Storyboard

- **Ziel des Spiels**

- Der Spieler muss aus der Einrichtung Heidelberg Dynamics fliehen
- Gegner besiegen
- Hindernisse überwinden

- Wie man spielt

- Gegnertypen

- Grafik und Audio

- Geschichte des Spiels



Storyboard

- Ziel des Spiels
- **Wie man spielt**
 - Bewegungssteuerung durch "WASD"
 - Zielen und schießen mit der Maus
 - Interagieren mit Umgebung mit "E"
- Gegnertypen
- Grafik und Audio
- Geschichte des Spiels



Storyboard

- Ziel des Spiels
- Wie man spielt
- **Gegnertypen**
 - “Turret”: einfaches Geschütz, durchschnittliche Feuerrate und Trefferpunkte
 - “Bossturret”: großes Geschütz, mit hoher Feuerrate und vielen Trefferpunkten
- Grafik und Audio
- Geschichte des Spiels



Storyboard

- Ziel des Spiels
- Wie man spielt
- Gegnertypen
- **Grafik und Audio**
 - Grafik im Pixel Style
 - Soundeffekte für verschiedene Situationen
 - Begleitende Hintergrundmusik
- Geschichte des Spiels

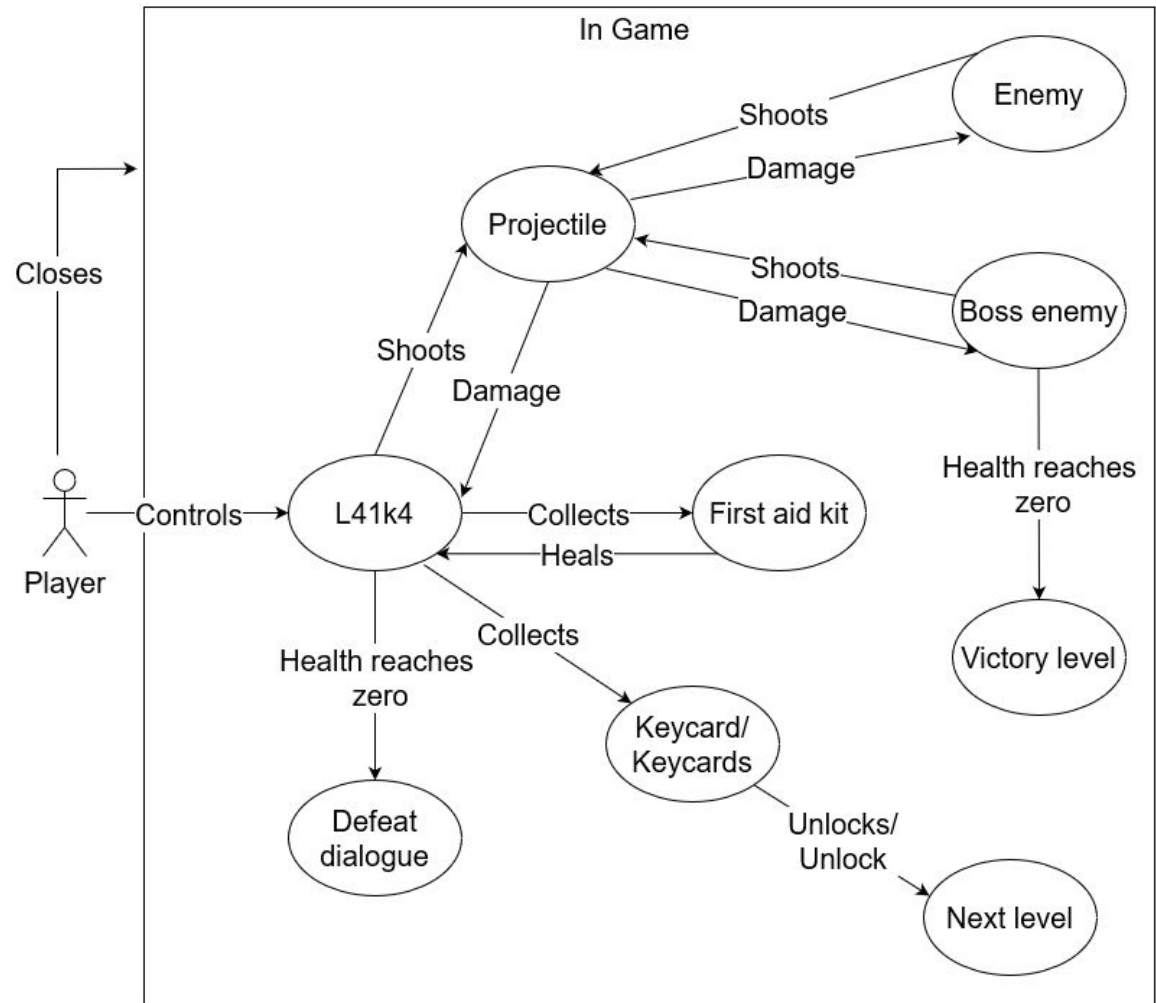


Storyboard

- Ziel des Spiels
- Wie man spielt
- Gegnertypen
- Grafik und Audio
- **Geschichte des Spiels**
 - Protagonist: Die Roboterhündin “L41k4”
 - Schauplatz: Forschungseinrichtung “Heidelberg Dynamics”



Use Case Diagramm



Anforderungsanalyse

- **Movement**
 - The player is required to be able to move the playable character throughout the levels.
- **Aim**
 - The player character should be able to aim using the mouse.
- **Interaction with environment**
 - The player character should move throughout a level, interacting with the environment. e.g. bump into a wall, walk down a corridor, etc. (Collision).
- **Health System**
 - The game features a health system that manages all the damage and health variables of all entities in the game.
 - The playable character and the enemies have a certain number of hitpoints.
 - When the hitpoints of a entity run out, the character is eliminated.
- **Projectiles**
 - Projectiles get destroyed when colliding with another object (i.e. walls, shelves, etc.).
- **Weapon**
 - The game is required to have a useable weapon which the player can use to reduce the hitpoints of enemies.
- **Enemies**
 - There have to be different enemies.
 - The enemies can attack and damage the player.
 - The player must also be able to defeat these enemies with a certain amount of attacks.
- **Keycards**
 - There are keycards the player can collect in each level.
 - To complete a level the player must be able to collect the keycards first.
- **End of the game**
 - The player can win the game by finishing all levels.
 - The player can lose the game when their hitpoints reach zero

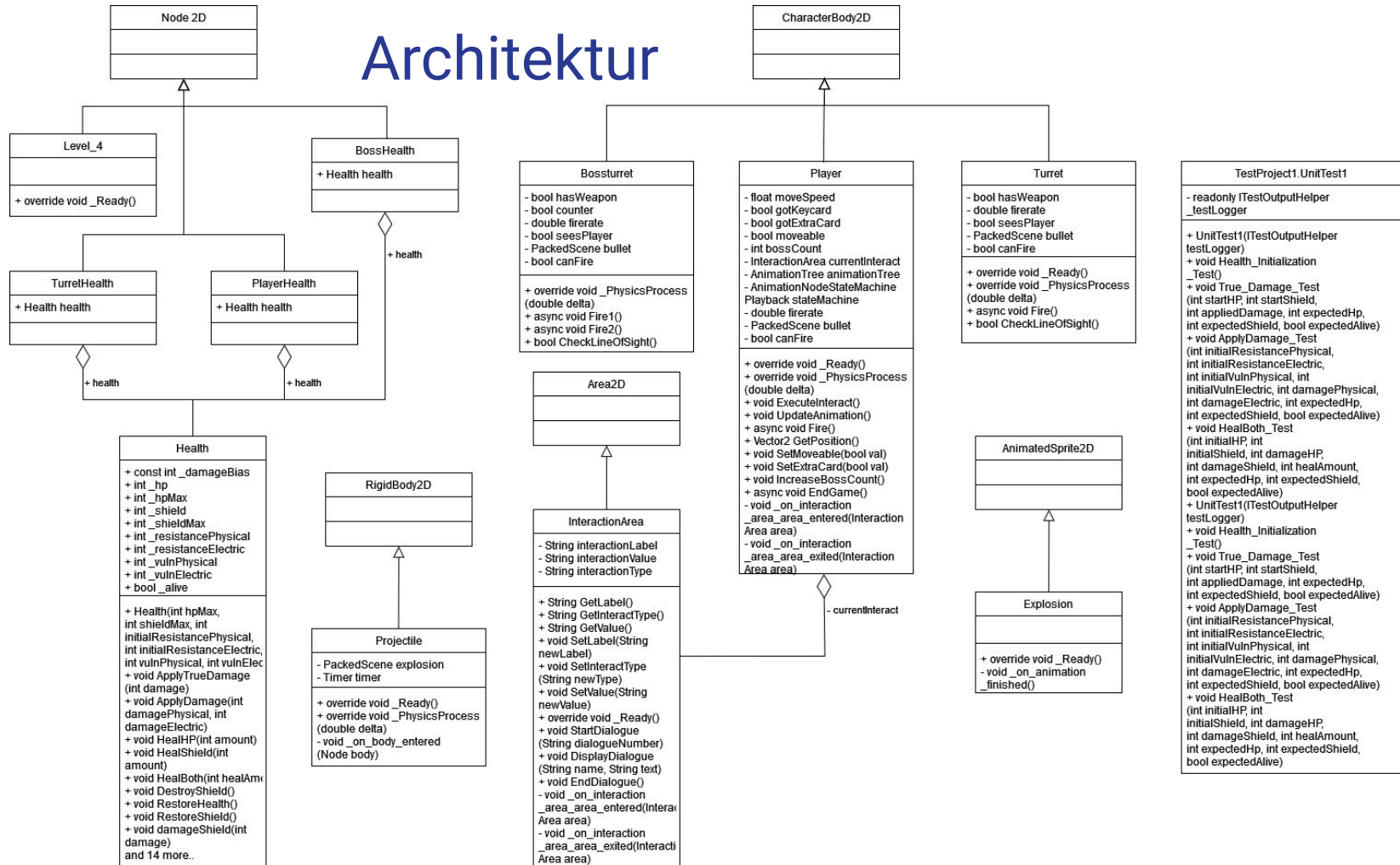
Anforderungsanalyse - verfeinert

- Movement
 - The player is required to be able to move the playable character throughout the levels with a fixed movement speed.
 - WASD for up, down, left, right.
- Aim
 - The player character is able to aim using the mouse.
 - The character aims in the cursor direction at all times.
- Interaction with environment
 - The player character should move throughout a level, interacting with the environment. e.g. bump into a wall, walk down a corridor, etc. (Collision).
- Health System
 - The game features a health system that manages all the damage and health variables of all entities in the game.
 - The playable character and the enemies have a certain number of hitpoints.
 - When the hitpoints of a entity run out, the character is eliminated.
 - The health and shield of the entities is limited between two values that are immutable.
 - There are two types of damage each entity type has: physical and electric.
 - There are resistance and vulnerability stats for each entity for the different damage types.
- Projectiles
 - Projectiles have a limited range. This is realized by a timer that destroys the bullet when it runs out.
 - Projectiles get destroyed when colliding with another object (i.e. walls, shelves, etc.).
 - When two projectiles collide they cancel each other out.
 - All projectiles travel with a fixed speed in a straight line.

Anforderungsanalyse - verfeinert

- **Weapon**
 - The game is required to have a useable weapon. The player can use the weapon to fire projectiles that reduce the hitpoints of enemies.
 - The weapon has a fixed firerate.
- **Enemies**
 - There have to be different enemies.
 - The enemies can attack and damage the player.
 - The player must also be able to defeat these enemies with a certain amount of attacks.
 - There is a enemy type "turret" that aims at the player and shoots projectiles at them.
 - There is a enemy type "bossturret" that aims at the player and shoots projectiles at them but at a significant faster pace than the normal "turret".
 - The enemies aim at the location of the player at all times.
 - The enemies shoot at a fixed firerate.
 - The turrets start shooting at the player as soon as they enter a certain range.
- **Keycards**
 - There are keycards the player can collect in each level.
 - To complete a level the player must be able to collect the keycards first.
- **First Aid Kits**
 - There are first aid kits that replenish the player's health and shield.
 - First aid kits are collected as soon as the player enters their collision zone
- **End of the game**
 - The player can win the game by defeating the two bossturret enemies in the last level.
 - The player can lose the game when their hitpoints reach zero.

Architektur



Implementation - Player Klasse

- PhysicsProcess - Funktion:
 - Wird jeden Tick des Spiels aufgerufen
 - Reagiert auf Input des Spielers
 - Aktualisiert Position des spielbaren Charakters oder ruft andere Interaktionen auf
- Speichert Fortschritte im Spiel:
 - gesammelte Keycards
 - besiegte Bossgegner
 - aktuelle Statusattribute wie Trefferpunkte und Bewegungsgeschwindigkeit



Implementation - InteractionArea

- Jede Entität im Spiel besitzt eine InteractionArea
- Löst beim Betreten automatisch Funktionen aus
- Verschiedene Instanzen:
 - Dialog:
 - stoppt bei Betreten das Spielgeschehen und öffnet ein Dialogfenster
 - Keycard:
 - weist bei Betreten den Spieler auf die Möglichkeit hin die Keycard aufzusammeln
 - ermöglicht Aufsammeln der Keycard bei Interaktion
 - First aid kit:
 - heilt Spieler unmittelbar beim Betreten um einen bestimmten Betrag
 - Levelübergang:
 - Durch Interaktion mit Tür wird bei Besitz der Keycards das nächste Level geladen



Implementation - Health

- Der Spieler und jeder Gegner besitzen eine Instanz dieser Klasse
- verwaltet und aktualisiert Trefferpunkte
- berechnet zugefügten Schaden unter Berücksichtigung der Resistenzwerte
- heilt Spieler beim Einsammeln eines First Aid Kits und dem Betreten eines neuen Levels
- löst bei Tod des Spielers das Ende des Spiels aus

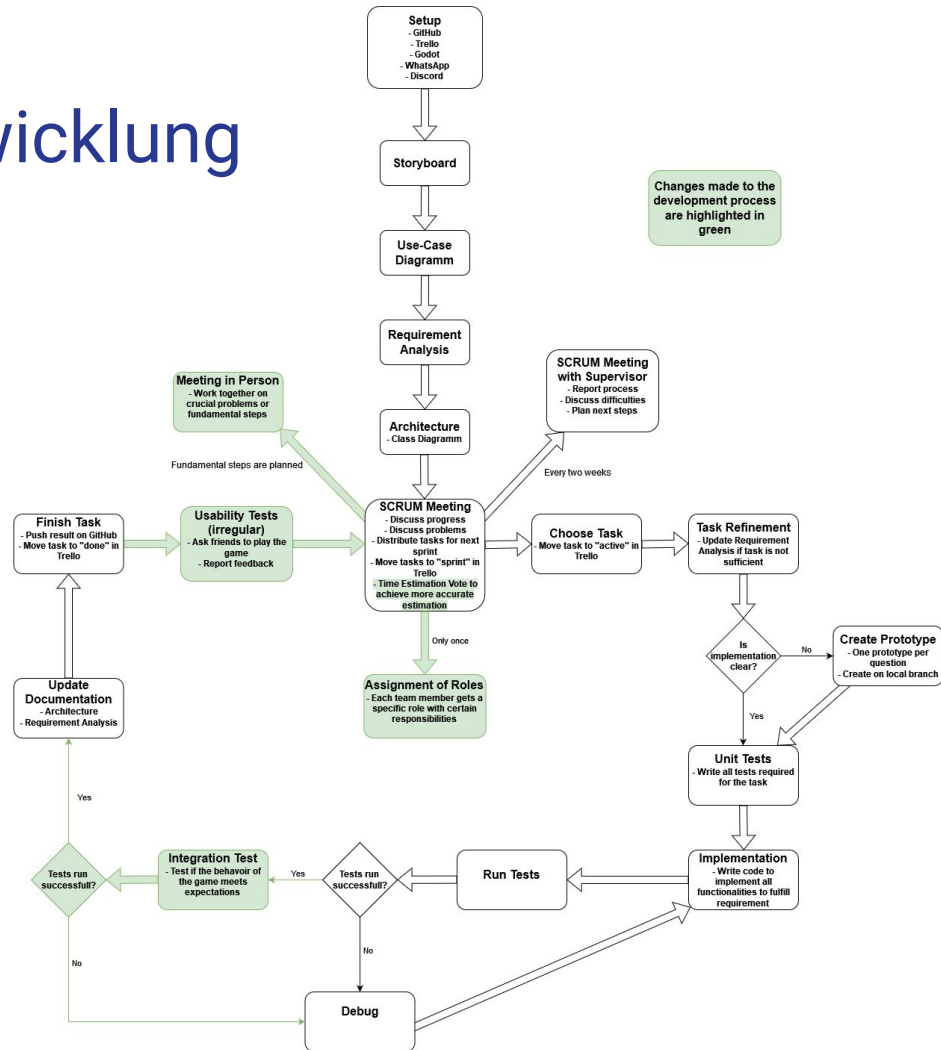


Implementation - Gegner

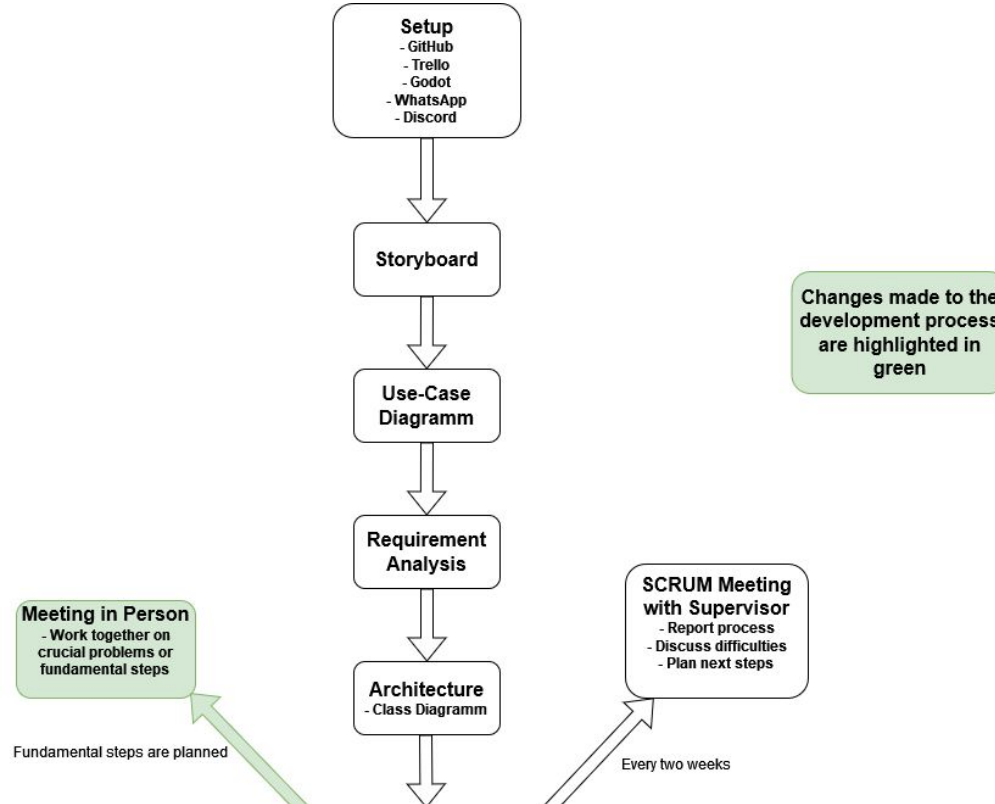
- Stationäre Geschütze, die Projektile auf den Spieler abfeuern, wenn dieser in Reichweite ist
- Besitzen Attribute wie Feuerrate, Trefferpunkte und Resistenzen
- Überprüfen durchgängig ob der Spieler in Reichweite ist:
 - Berechnen der Distanz zum Spieler aus Differenz der eigenen Position und Position des Spielers



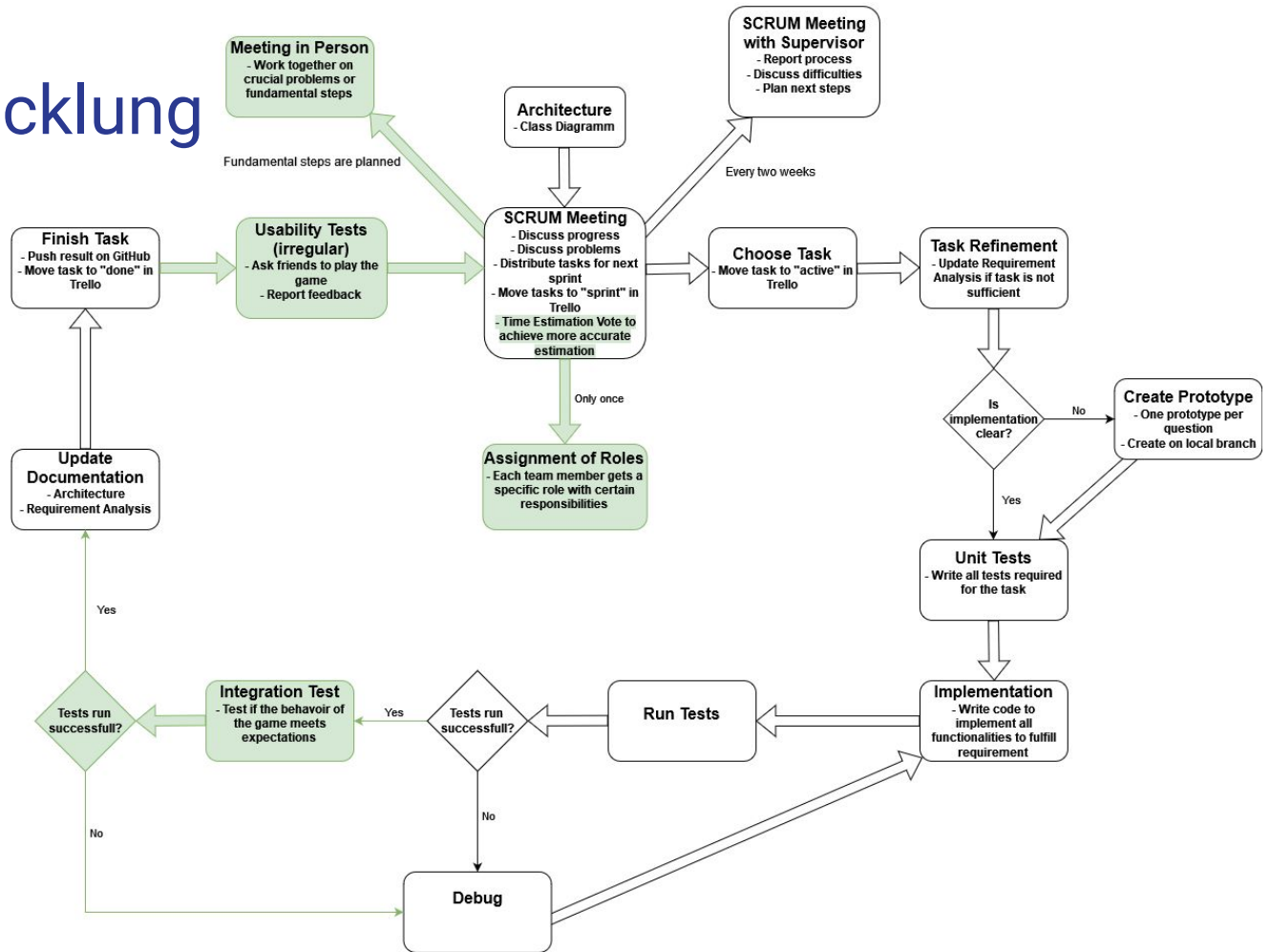
Prozessentwicklung



Prozessentwicklung



Prozessentwicklung

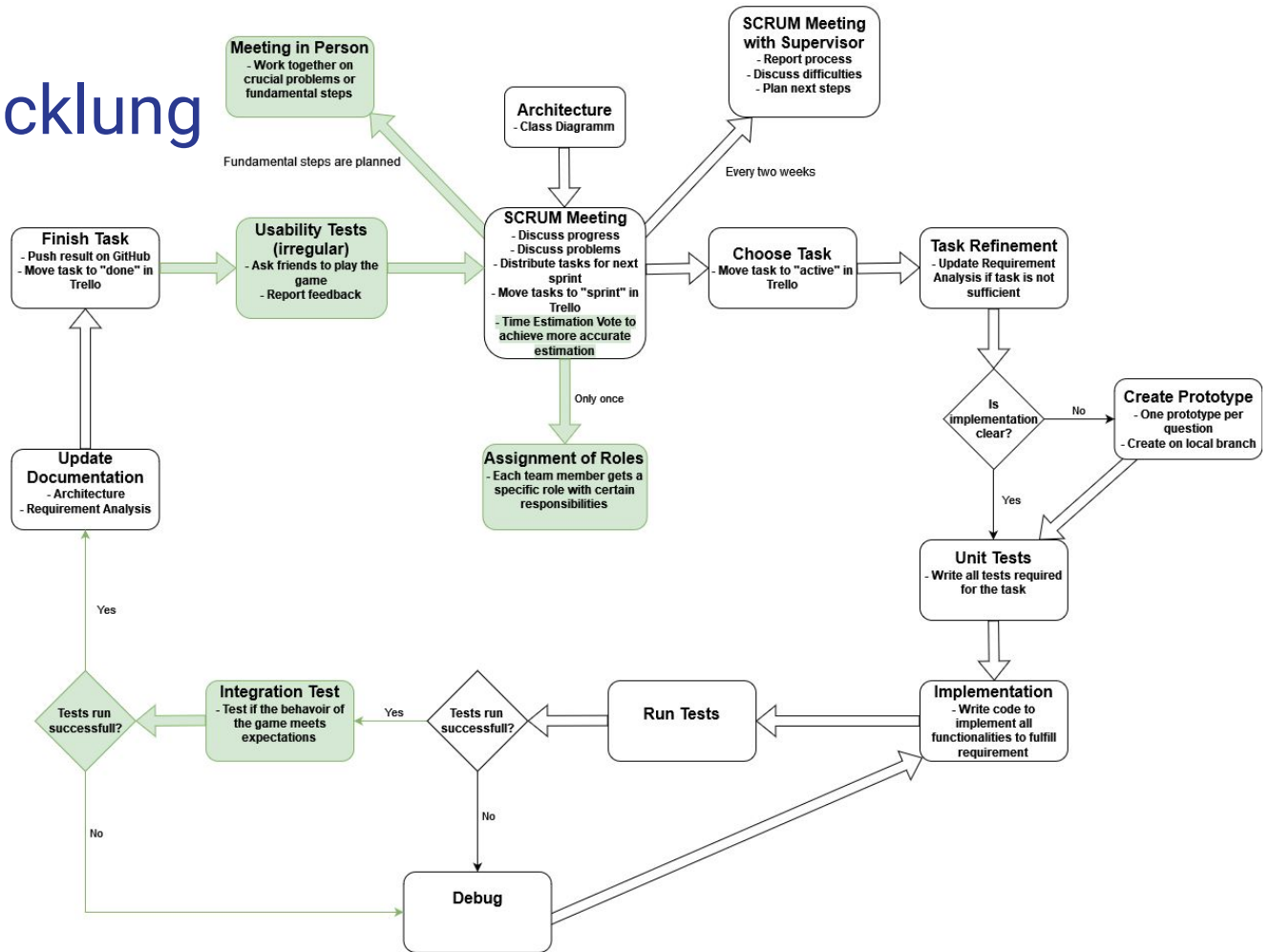


Prozess - Test-Driven Development (TDD)

- Essentiell für fehlerfreie Entwicklung von Code
- Zwingt Entwickler sich vor dem Schreiben des Codes intensiv mit der Problemstellung zu befassen
- Anfangs Schwierigkeiten beim Testen in Godot
- Nach Erstellen von Subsystemen in Visual Studio Code konnte TDD in unseren Prozess integriert werden
- Trotz intensiver Planung, dennoch herausfordernd alle Testfälle abzudecken



Prozessentwicklung

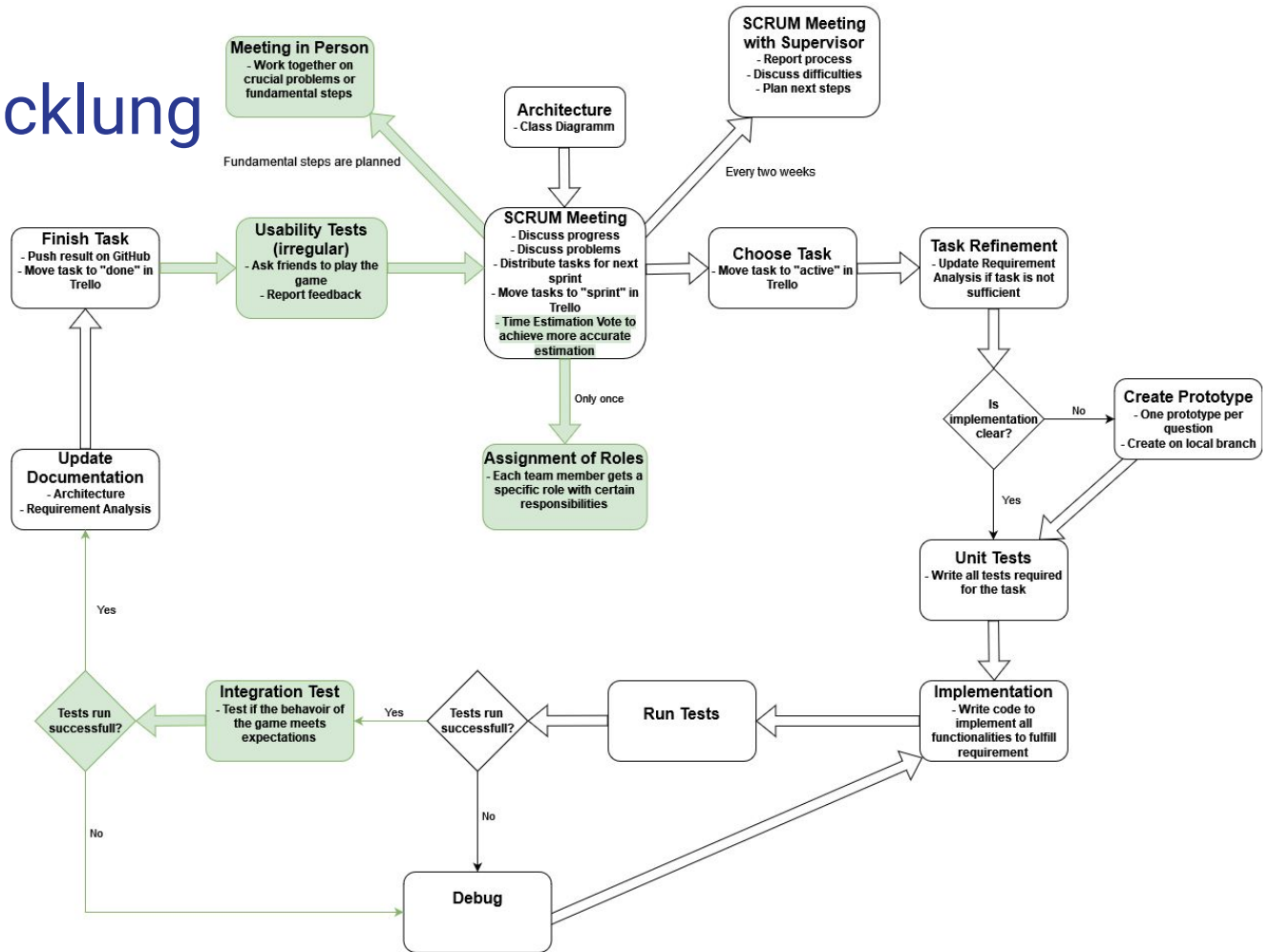


Prozess - Prototyping

- Notwendig um fundamentale Bestandteile der Engine zu verstehen
- Wurde an vielen Stellen eingesetzt um Konzepte der Engine auszutesten
- Wichtig um sich vor Augen zu führen, wie bestimmte Anforderungen umzusetzen sind



Prozessentwicklung

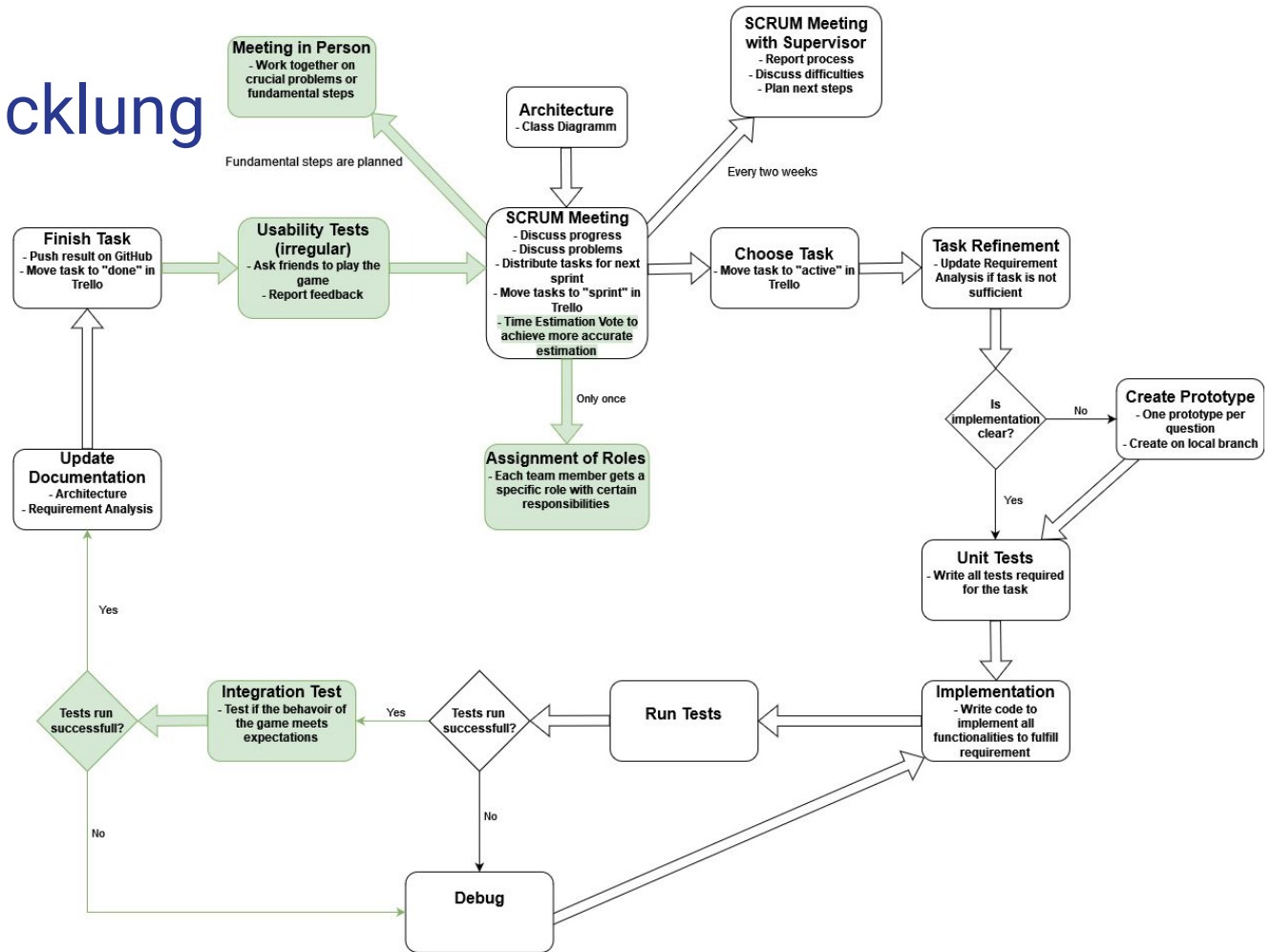


Prozessentwicklung Rollenzuweisung

- Anfangs unklar welches Teammitglied, welche Aufgaben zu erfüllen hat
- Rollen
 - Erstellen und Warten von technischen Tools, wie beispielsweise Versionskontrolle (GitHub, Trello, ...)
 - Planen von Raum und Zeit der teaminternen (SCRUM-) Meetings und Erstellen sowie Warten von Tools zur Kommunikation (Discord, Whatsapp)
 - Leiten und Protokollieren der teaminternen SCRUM-Meetings und übernimmt die Position des Teamleiters
- Nach Umsetzung: Arbeit im Team strukturierter und effektiver



Prozessentwicklung

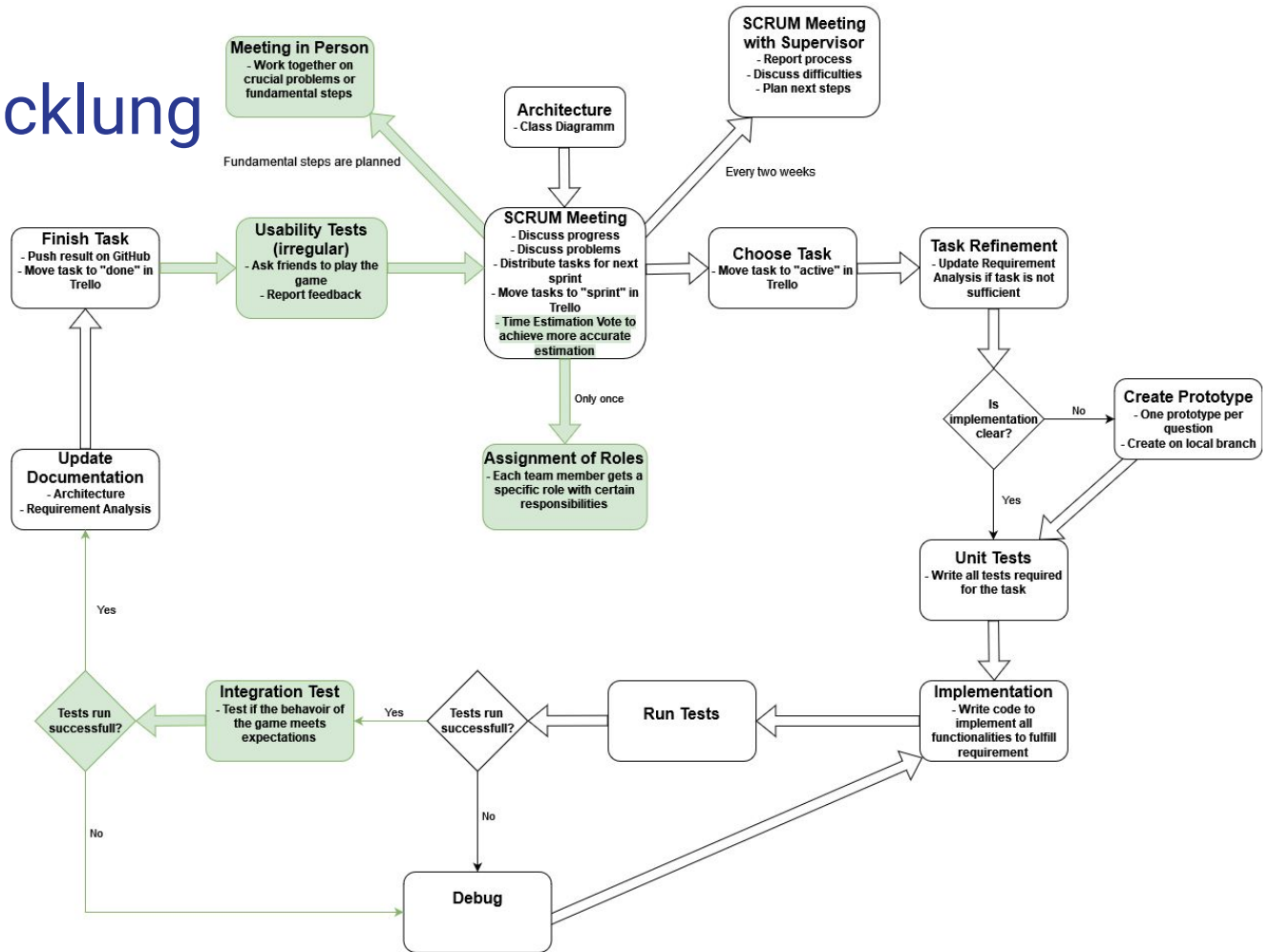


Prozessentwicklung - Meetings in Person

- Alle Teammitglieder müssen bestimmte essentielle Konzepte und Grundlagen von Godot verstehen
- Meetings in Person um zusammen an derartigen fundamentalen Problemen zu arbeiten
- Bei SCRUM-Meeting planen, ob ein solches Meeting für die nächsten Schritte notwendig ist
- Nach Umsetzung:
 - Erlernen von Konzepten der Engine leichter und schneller durch Austausch unter den Teammitgliedern
 - Stellt sicher, dass alle Teammitglieder auf dem selben Stand sind



Prozessentwicklung

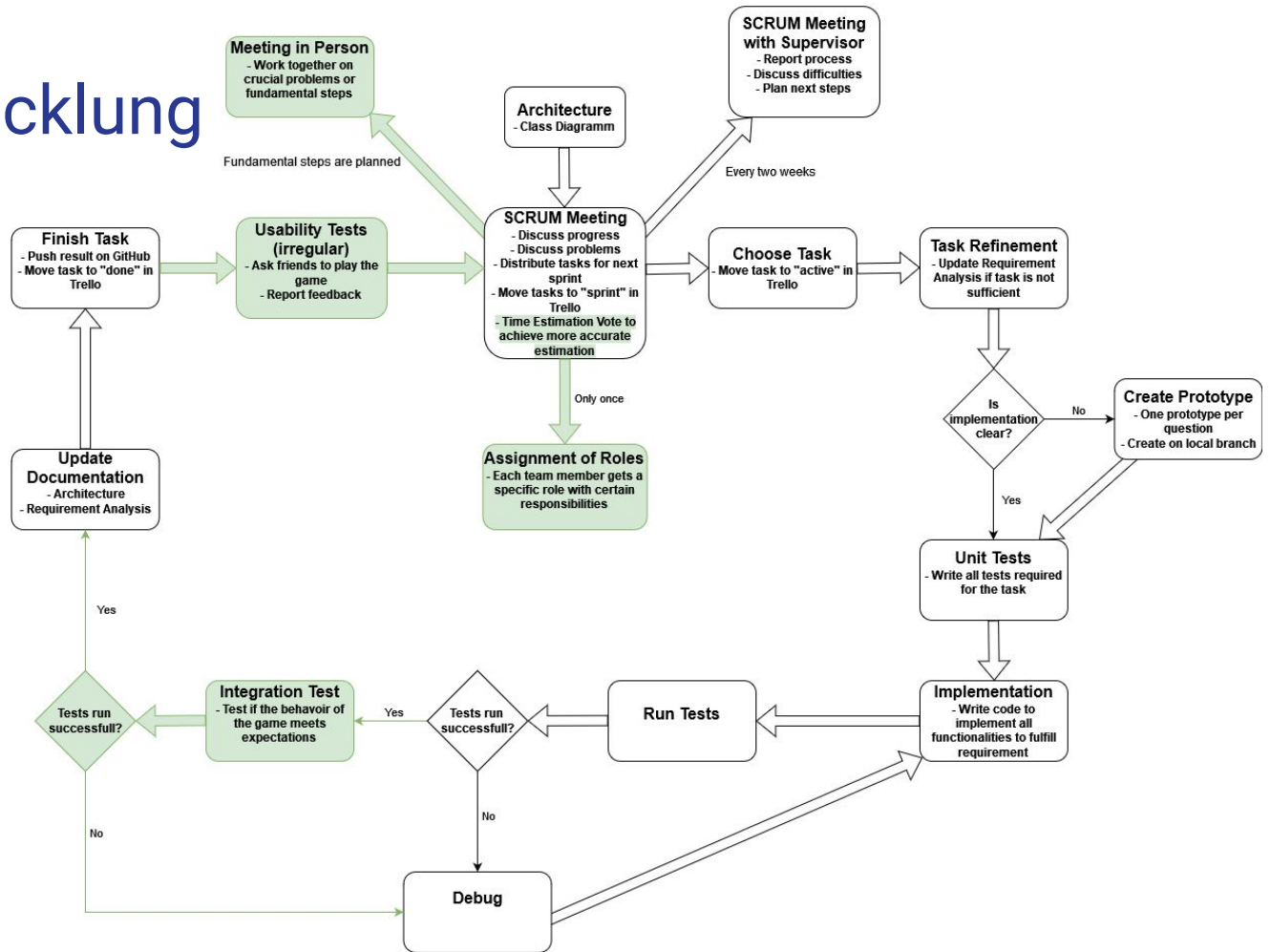


Prozessentwicklung - Abstimmung zum Zeitaufwand

- Zeitaufwand für Anforderungen oft nur schwer einzuschätzen
- Einführen einer Abstimmung zum Zeitaufwand im Team
- Nach Umsetzung:
 - Miteinbeziehen aller Teammitglieder
 - Genauere Einschätzungen möglich
 - Ermöglicht bessere Aufgabenverteilung



Prozessentwicklung

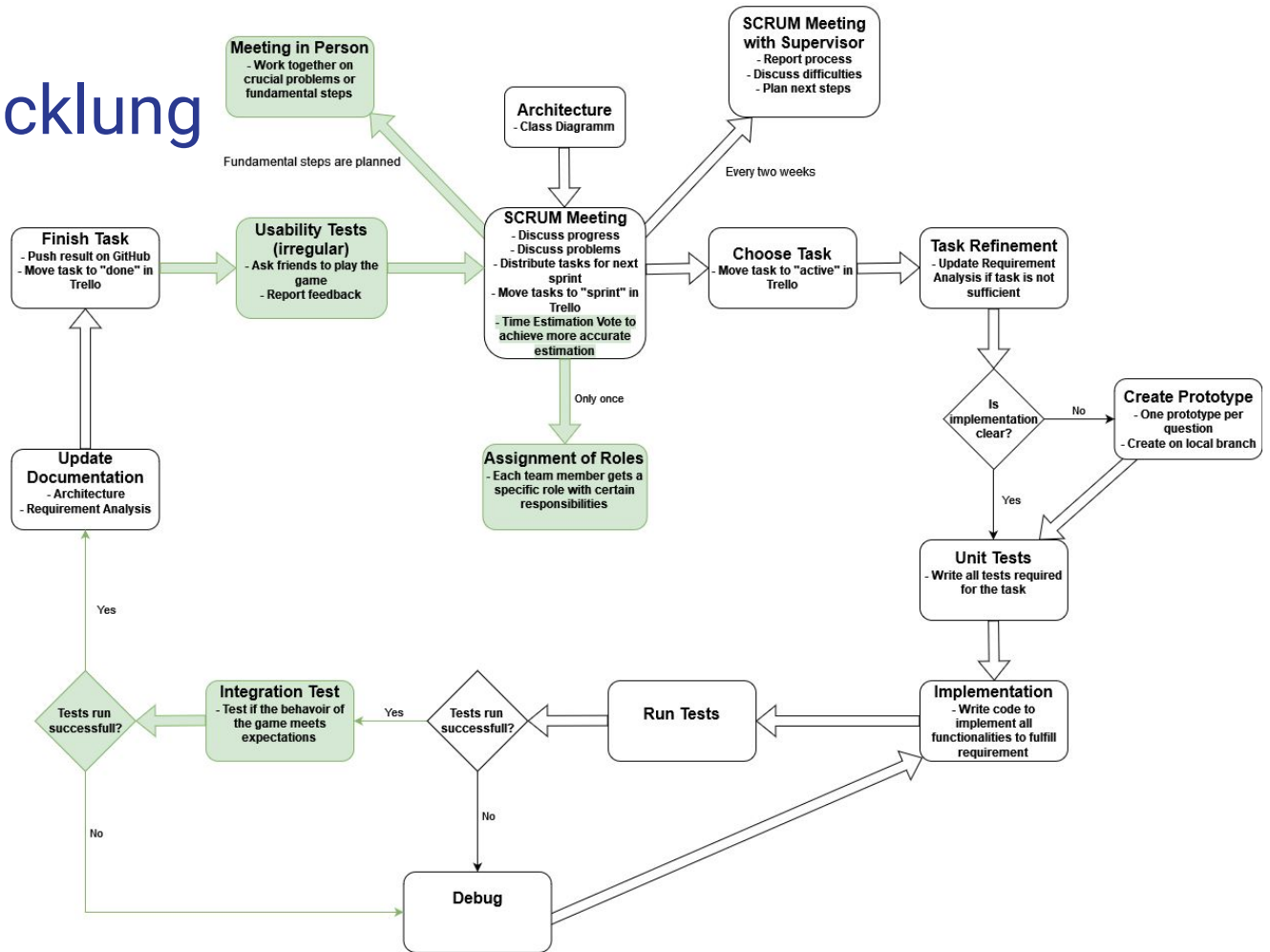


Prozessentwicklung - Einführen von Integrationstest

- TDD sehr hilfreich für fehlerfreie Codeentwicklung, aber oftmals weicht erwartetes Verhalten von tatsächlichen Verhalten ab, trotz bestandener Tests
- Zusätzliche Integrationstest, die jedes Teammitglied durchführt bevor Code auf GitHub gepusht wird
- Nach Umsetzung:
 - Stellt sicher, dass Code tatsächlich den Erwartungen entspricht
 - Verhindern vieler möglicher Fehler in der Entwicklung



Prozessentwicklung



Prozessentwicklung - Usability Test mit Kommilitonen

- Nach längerer Entwicklung sind Teammitglieder sehr vertraut mit Spiel, dadurch Einschätzung des Schwierigkeitsgrads verzerrt
- Hinzuziehen von Dritten, die das Spiel testen in unregelmäßigen Abständen
- Nach Umsetzung:
 - Feedback der Spieler ermöglicht präzise Verbesserung der Spielerfahrung
 - Beobachten der Spieler kann zum Entdecken von Fehlern im Spiel führen
 - Zeigt neue Ideen für das Spiel auf und lässt Prioritäten bei der Entwicklung anpassen



Zusammenfassung

- Während des Praktikums kam es zu vielen Schwierigkeiten, die unseren Fortschritt hinderten
 - Durch die Weiterentwicklung unseres Entwicklungsprozesses gelang es uns diese Schwierigkeiten zu überwinden
 - Besonders die Meetings in Person und die Integrationstests waren erhebliche Verbesserungen
 - Wir sind zuversichtlich, dass uns dadurch zukünftige Projekte erheblich leichter fallen werden, was einen großen Erfolg des Praktikums für uns markiert
- 